Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Навчальний проєкт №2

Методи і технології паралельного програмування

Тема: Створення розподіленої паралельної системи обміну повідомленнями

Виконав: Перевірив:

студент групи ІП-51мн Лесик В. О.

Панченко С. В.

Київ 2026

ЗМІСТ

1 Мета роботи 3
2 Завдання до виконання 4
3 Обґрунтування обрання фреймворку 5
4 Опис рішень та критичних моментів 6
4.1 Задача 1. Розділення каналів читання та запису 6
4.2 Задача 2. Маршрутизація повідомлень 6
4.3 Задача 3. Доступ до спільного стану та уникнення взаємних блокувань .6
5 Результати роботи програми 7
6 Висновки 8
А Лістинг коду 9
А.1 src/bin/server.rs 9
А.2 src/bin/client.rs 14

1 МЕТА РОБОТИ

Засвоїти принципи побудови розподілених клієнт-серверних систем обміну повідомленнями. Навчитися використовувати інструменти паралельного програмування для одночасного обслуговування багатьох клієнтів. Дослідити проблеми синхронізації, конкурентного доступу до спільних ресурсів та мережевої взаємодії.

2 ЗАВДАННЯ ДО ВИКОНАННЯ

1. Розробити працюючий сервер та клієнт з використанням сокетів.
2. Забезпечити одночасну роботу декількох клієнтів без блокування системи одним повільним з’єднанням.
3. Реалізувати підтримку індивідуальних повідомлень та спілкування в цільових групах.
4. Забезпечити ведення списку активних учасників та коректну обробку відключень клієнтів від сервера.

3 ОБҐРУНТУВАННЯ ОБРАННЯ ФРЕЙМВОРКУ

Для реалізації проєкту обрано мову Rust та асинхронне середовище виконання Tokio. Використання системних потоків для кожного мережевого з’єднання є ресурсомістким підходом. Tokio використовує модель асинхронного вводу-виводу та зелені потоки, які мультиплексуються на невеликий пул системних потоків. Це дозволяє серверу ефективно масштабуватися та обслуговувати тисячі одночасних підключень з мінімальними накладними витратами пам’яті та процесорного часу.

4 ОПИС РІШЕНЬ ТА КРИТИЧНИХ МОМЕНТІВ

4.1 Задача 1. Розділення каналів читання та запису

Для одночасної обробки вхідних повідомлень від клієнта та надсилання йому повідомлень від інших користувачів, TCP-сокет було розділено на дві незалежні частини. Використання методу `into_split()` дозволило передати право власності на частину запису у виділений асинхронний потік, тоді як головний цикл підключення обробляє читання. Це вирішило проблему тривалості життя об'єктів у багатопотоковому середовищі Rust.

4.2 Задача 2. Маршрутизація повідомлень

Замість простого широкомовного розсилання було використано багатоканальну модель зв'язку. Для кожного клієнта створюється власний канал передачі даних. Сервер зберігає вказівники відправки для кожного користувача у спільному словнику. Це дозволяє точно маршрутизувати повідомлення конкретному адресату або ітеруватися по членах групи для групового розсилання.

4.3 Задача 3. Доступ до спільного стану та уникнення взаємних блокувань

Дані про активних користувачів та їх належність до груп зберігаються у структурі `AppState`, яка захищена механізмом взаємного виключення для потокобезпечного конкурентного доступу. Критичним моментом було уникнення конфліктів при одночасному мутуванні стану та ітерації по ньому. Проблему вирішено шляхом розділення операцій на ізольовані області видимості, що гарантує звільнення блокування до початку розсилки по мережі.

5 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

```
user@disco:~/university/masters_second_semester_methods_and_technologies_of_parallel_programming/pr2$ cargo run --release --bin server
   Compiling pr2 v0.1.0 (/home/user/university/masters_second_semester_methods_and_technologies_of_parallel_programming/pr2)
    Finished `release` profile [optimized] target(s) in 0.88s
     Running `target/release/server`
Server started on 127.0.0.1:8080
```

Рисунок 5.1 — Логи термінала сервера під час роботи клієнтів

```
cargo run --release --bin client
 --- Distributed Chat System ---
Logged in as: sonic
Commands:
  <text>                - Sends to 'global' group
  /join <group>         - Join a new or existing group
  /leave <group>        - Leave a group
  /list <group>         - View active members in a group
  /g <group> <text>     - Message a specific group
  /msg <user> <text>    - Send a private direct message
==========================================================
** System: shadow joined global **
** System: emmy joined global **
[Group: global] sonic: hell
[Group: global] sonic: whoah
[Group: global] shadow: mama mia
** System: Users in global: sonic, emmy, shadow **
[Group: global] sonic: join diablo1
** System: You are not a member of diablo1 **
** System: You joined diablo1 **
[Group: diablo1] sonic: ahah i ma here
[Group: global] sonic: df
[Group: global] sonic: fd
[Private from emmy] hello dear )))) -_-
[Group: diablo1] shadow: hello from shadow, sonic does not know
** System: Users in diablo2: shadow, emmy **
 >
```

```
cargo run --release --bin client
 --- Distributed Chat System ---
Logged in as: shadow
Commands:
  <text>                - Sends to 'global' group
  /join <group>         - Join a new or existing group
  /leave <group>        - Leave a group
  /list <group>         - View active members in a group
  /g <group> <text>     - Message a specific group
  /msg <user> <text>    - Send a private direct message
==========================================================
** System: emmy joined global **
[Group: global] sonic: hell
[Group: global] sonic: whoah
[Group: global] shadow: mama mia
[Group: global] sonic: join diablo1
** System: You joined diablo1 **
** System: sonic joined diablo1 **
[Group: diablo1] sonic: ahah i ma here
[Group: global] sonic: df
[Group: global] sonic: fd
 > /g diablo2 hello from shadow, sonic does now** System: You
are not a member of diablo2 **
 > /g diablo1 hello from shadow, sonic does not[Group: diablo1]
 shadow: hello from shadow, sonic does not know
** System: You joined diablo2 **
[Group: diablo2] shadow: sonic is not here ))))
** System: Users in diablo1: shadow, sonic **
 >
```

```
cargo run --release --bin client
 --- Distributed Chat System ---
Logged in as: emmy
Commands:
  <text>                - Sends to 'global' group
  /join <group>         - Join a new or existing group
  /leave <group>        - Leave a group
  /list <group>         - View active members in a group
  /g <group> <text>     - Message a specific group
  /msg <user> <text>    - Send a private direct message
==========================================================
[Group: global] sonic: hell
[Group: global] sonic: whoah
[Group: global] shadow: mama mia
[Group: global] sonic: join diablo1
[Group: global] sonic: df
[Group: global] sonic: fd
** System: You joined diablo2 **
[Private to sonic] hello dear )))) -_-
** System: shadow joined diablo2 **
[Group: diablo2] shadow: sonic is not here ))))
 >
```

Рисунок 5.2 — Інтерфейс клієнта: приватні та групові повідомлення

6 ВИСНОВКИ

Під час виконання проєкту було розроблено розподілену клієнт-серверну чат-систему. Використання асинхронної моделі на базі Rust та Tokio довело свою ефективність для завдань, що інтенсивно використовують мережевий ввід-вивід. Реалізовано надійний механізм точкової маршрутизації повідомлень за допомогою багатоканальної взаємодії потоків. Вирішено класичні проблеми багатопотоковості: безпечний доступ до спільної пам’яті реалізовано через механізми взаємного виключення, а стан гонитви та можливі взаємні блокування усунуто завдяки правильному управлінню тривалістю утримання блокувань. Програма успішно підтримує як глобальний чат, так і приватні та групові канали зв’язку.

Додаток А
Лістинг коду

A.1 src/bin/server.rs

```
use std::collections::{HashMap, HashSet};
use std::sync::Arc;
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::net::TcpListener;
use tokio::sync::{Mutex, mpsc};

struct AppState {
    clients: HashMap<String, mpsc::Sender<String>>,
    groups: HashMap<String, HashSet<String>>,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let listener = TcpListener::bind("127.0.0.1:8080").await?;
    println!("Server started on 127.0.0.1:8080");

    let state = Arc::new(Mutex::new(AppState {
        clients: HashMap::new(),
        groups: HashMap::new(),
    }));

    state
        .lock()
        .await
        .groups
        .insert("global".to_string(), HashSet::new());

    loop {
        let (socket, _addr) = listener.accept().await?;
        let state = Arc::clone(&state);

        tokio::spawn(async move {
            let (reader, mut writer) = socket.into_split();
            let mut reader = BufReader::new(reader);
            let mut nickname = String::new();

            if reader.read_line(&mut nickname).await.unwrap_or(0) == 0
{
                return;
            }
            let nickname = nickname.trim().to_string();

            let (tx, mut rx) = mpsc::channel::<String>(100);
```

```
            {
                let mut st = state.lock().await;
                if st.clients.contains_key(&nickname) {
                    let _ = writer.write_all(b"TAKEN\n").await;
                    return;
                }

                st.clients.insert(nickname.clone(), tx.clone());
                st.groups
                    .get_mut("global")
                    .unwrap()
                    .insert(nickname.clone());

                let sys_msg = format!("*** System: {} joined global
***\n", nickname);
                for member in st.groups.get("global").unwrap() {
                    if member != &nickname {
                        if let Some(member_tx) = st.clients.get(member)
{
                            let _ =
member_tx.send(sys_msg.clone()).await;
                        }
                    }
                }
            }
            let _ = writer.write_all(b"OK\n").await;

            tokio::spawn(async move {
                while let Some(msg) = rx.recv().await {
                    if writer.write_all(msg.as_bytes()).await.is_err()
{
                        break;
                    }
                }
            });

            let mut line = String::new();
            while let Ok(n) = reader.read_line(&mut line).await {
                if n == 0 {
                    break;
                }
                let input = line.trim();
                if input.is_empty() {
                    continue;
                }

                let mut st = state.lock().await;
```

```
                let parts: Vec<&str> = input.splitn(3, ' ').collect();
                let cmd = parts[0];

                match cmd {
                    "/join" if parts.len() == 2 => {
                        let group = parts[1].to_string();
                        st.groups
                            .entry(group.clone())
                            .or_insert_with(HashSet::new)
                            .insert(nickname.clone());
                        let sys_msg = format!("*** System: {} joined {}
***\n", nickname, group);

                        for member in st.groups.get(&group).unwrap() {
                            if let Some(member_tx) =
st.clients.get(member) {
                                let _ = member_tx
                                    .send(if member == &nickname {
                                        format!("*** System: You joined
{} ***\n", group)
                                    } else {
                                        sys_msg.clone()
                                    })
                                    .await;
                            }
                        }
                    }
                    "/leave" if parts.len() == 2 => {
                        let group = parts[1];
                        if group == "global" {
                            let _ = tx
                                .send(
                                    "*** System: You cannot leave the
global group ***\n"
                                        .to_string(),
                                )
                                .await;
                        } else {
                            let mut left_successfully = false;
                            if let Some(members) =
st.groups.get_mut(group) {
                                left_successfully =
members.remove(&nickname);
                            }
                            if left_successfully {
                                let _ = tx
                                    .send(format!("*** System: You left
{} ***\n", group))
```

```
                                    .await;
                                let sys_msg =
                                    format!("*** System: {} left {}
***\n", nickname, group);

                                if let Some(members) =
st.groups.get(group) {
                                    for member in members.iter() {
                                        if let Some(member_tx) =
st.clients.get(member) {
                                            let _ =
member_tx.send(sys_msg.clone()).await;
                                        }
                                    }
                                }
                            }
                        }
                    }
                    "/list" if parts.len() == 2 => {
                        let group = parts[1];
                        if let Some(members) = st.groups.get(group) {
                            let users: Vec<String> =
members.iter().cloned().collect();
                            let _ = tx
                                .send(format!(
                                    "*** System: Users in {}: {}
***\n",
                                    group,
                                    users.join(", ")
                                ))
                                .await;
                        } else {
                            let _ = tx
                                .send(format!("*** System: Group {}
does not exist ***\n", group))
                                .await;
                        }
                    }
                    "/g" if parts.len() == 3 => {
                        let group = parts[1];
                        let msg = parts[2];
                        if let Some(members) = st.groups.get(group) {
                            if members.contains(&nickname) {
                                let formatted =
                                    format!("[Group: {}] {}: {}\n",
group, nickname, msg);
                                for member in members.iter() {
                                    if let Some(member_tx) =
```

```
st.clients.get(member) {
                                        let _ =
member_tx.send(formatted.clone()).await;
                                    }
                                }
                            } else {
                                let _ = tx
                                    .send(format!(
                                        "*** System: You are not a
member of {} ***\n",
                                        group
                                    ))
                                    .await;
                            }
                        }
                    }
                    "/msg" if parts.len() == 3 => {
                        let target = parts[1];
                        let msg = parts[2];
                        if target == nickname {
                            let _ = tx.send("*** System: You cannot
send a private message to yourself ***\n".to_string()).await;
                        } else if let Some(target_tx) =
st.clients.get(target) {
                            let _ = target_tx
                                .send(format!("[Private from {}] {}\n",
nickname, msg))
                                .await;
                            let _ = tx.send(format!("[Private to {}]
{}\n", target, msg)).await;
                        } else {
                            let _ = tx
                                .send(format!("*** System: User {} is
not online ***\n", target))
                                .await;
                        }
                    }
                    _ => {
                        let msg = input;
                        let formatted = format!("[Group: global] {}:
{}\n", nickname, msg);
                        if let Some(members) = st.groups.get("global")
{
                            for member in members.iter() {
                                if let Some(member_tx) =
st.clients.get(member) {
                                    let _ =
member_tx.send(formatted.clone()).await;
```

```
                                }
                            }
                        }
                    }
                }
                line.clear();
            }

            let mut st = state.lock().await;
            st.clients.remove(&nickname);

            let mut affected_groups = Vec::new();
            for (group_name, members) in st.groups.iter_mut() {
                if members.remove(&nickname) {
                    affected_groups.push(group_name.clone());
                }
            }

            for group in affected_groups {
                let sys_msg = format!(
                    "*** System: {} left {} (Disconnected) ***\n",
                    nickname, group
                );
                if let Some(members) = st.groups.get(&group) {
                    for member in members.iter() {
                        if let Some(tx) = st.clients.get(member) {
                            let _ = tx.send(sys_msg.clone()).await;
                        }
                    }
                }
            }
        });
    }
}
```

A.2 src/bin/client.rs

```
use crossterm::{
    cursor, execute,
    style::{self, Stylize},
    terminal,
};
use std::io::{Write, stdout};
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::net::TcpStream;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    print!("Enter your unique nickname: ");
```

```
    stdout().flush()?;
    let mut nickname = String::new();
    std::io::stdin().read_line(&mut nickname)?;

    let stream = TcpStream::connect("127.0.0.1:8080").await?;
    let (reader, mut writer) = stream.into_split();
    let mut reader = BufReader::new(reader);
    let mut stdin_reader = BufReader::new(tokio::io::stdin());

    writer.write_all(nickname.as_bytes()).await?;
    let mut auth_resp = String::new();
    reader.read_line(&mut auth_resp).await?;

    if auth_resp.trim() == "TAKEN" {
        println!(
            "{}",
            "Error: Nickname is active. Disconnecting.".red().bold()
        );
        return Ok(());
    }

    execute!(
        stdout(),
        terminal::Clear(terminal::ClearType::All),
        cursor::MoveTo(0, 0)
    )?;

    println!("{}", " --- Distributed Chat System --- ".bold().cyan());
    println!(
        "{}",
        format!("Logged in as: {}", nickname.trim())
            .italic()
            .dark_grey()
    );
    println!("{}", "Commands:".yellow());
    println!(
        "{}",
        "  <text>                - Sends to 'global' group".dark_grey()
    );
    println!(
        "{}",
        "  /join <group>         - Join a new or existing
group".dark_grey()
    );
    println!("{}", "  /leave <group>        - Leave a
group".dark_grey());
    println!(
        "{}",
```

```
        "  /list <group>         - View active members in a
group".dark_grey()
    );
    println!(
        "{}",
        "  /g <group> <text>     - Message a specific
group".dark_grey()
    );
    println!(
        "{}",
        "  /msg <user> <text>    - Send a private direct
message".dark_grey()
    );
    println!("{}", "=".repeat(60).dark_grey());

    loop {
        execute!(
            stdout(),
            cursor::MoveToColumn(0),
            terminal::Clear(terminal::ClearType::CurrentLine),
            style::Print(" > ".green().bold())
        )?;
        stdout().flush()?;

        let mut server_msg = String::new();
        let mut user_input = String::new();

        tokio::select! {
            res = reader.read_line(&mut server_msg) => {
                if res.unwrap_or(0) == 0 { break; }

                execute!(stdout(), cursor::MoveToColumn(0),
terminal::Clear(terminal::ClearType::CurrentLine))?;

                let msg = server_msg.trim();
                if msg.starts_with("*** System:") {
                    println!("{}", msg.yellow());
                } else if msg.starts_with("[Private") {
                    println!("{}", msg.magenta().bold());
                } else if msg.starts_with("[Group:") {
                    // Differentiate global from other groups visually
                    if msg.starts_with("[Group: global]") {
                        println!("{}", msg.white());
                    } else {
                        println!("{}", msg.cyan());
                    }
                } else {
                    println!("{}", msg);
```

```
                }
            }
            res = stdin_reader.read_line(&mut user_input) => {
                if res.is_ok() && !user_input.trim().is_empty() {
                    writer.write_all(user_input.as_bytes()).await?;
                    execute!(stdout(), cursor::MoveUp(1),
cursor::MoveToColumn(0),
terminal::Clear(terminal::ClearType::CurrentLine))?;
                }
            }
        }
    }

    execute!(
        stdout(),
        cursor::MoveToColumn(0),
        terminal::Clear(terminal::ClearType::CurrentLine)
    )?;
    println!("{}", "Disconnected from server.".red());
    Ok(())
}
```